README

pkg/userflow

A generic, multi-platform user flow testing framework for the `digital.vasic.challenges` module. It provides a unified interface for automating and validating user interactions across browser, mobile, desktop, API, and build pipelines -- all orchestrated through the challenge runner.

Package Purpose

`pkg/userflow` enables writing structured, reproducible test scenarios that span multiple platforms. Instead of scattering platform-specific test logic across different tools, this package defines a common adapter pattern where each platform implements a well-defined interface. Challenge templates consume these adapters to execute flows and produce assertion results compatible with the challenge framework.

Package Structure

```
pkg/userflow/
  -- Interfaces (adapter contracts)
  adapter_api.go           APIAdapter interface
  adapter_browser.go       BrowserAdapter interface
  adapter_build.go         BuildAdapter interface
  adapter_desktop.go       DesktopAdapter interface
  adapter_mobile.go        MobileAdapter interface
  adapter_process.go       ProcessAdapter interface

  -- Concrete implementations
  http_api_adapter.go      HTTPAPIAdapter (wraps
httpclient.APIClient)
  playwright_cli_adapter.go PlaywrightCLIAdapter (CDP via podman
exec)
  tauri_cli_adapter.go     TauriCLIAdapter (WebDriver protocol)
  adb_cli_adapter.go       ADBCLIAdapter (Android Debug Bridge)
  process_cli_adapter.go   ProcessCLIAdapter (subprocess
```

```
management)
  gradle_cli_adapter.go    GradleCLIAdapter (Gradle build
toolchain)
  npm_cli_adapter.go       NPMCLIAdapter (npm/npx build toolchain)
  go_cli_adapter.go        GoCLIAdapter (Go build toolchain)
  cargo_cli_adapter.go     CargoCLIAdapter (Cargo build toolchain)

  -- Flow definitions (step sequences)
  flow_api.go              APIFlow, APIStep, StepAssertion
  flow_browser.go          BrowserFlow, BrowserStep
  flow_mobile.go           MobileFlow, MobileStep
  flow_ipc.go              IPCCommand

  -- Challenge templates
  challenge_api_flow.go    APIHealthChallenge, APIFlowChallenge
  challenge_browser.go     BrowserFlowChallenge
  challenge_build.go       BuildChallenge, UnitTestChallenge,
LintChallenge
  challenge_desktop.go     DesktopLaunchChallenge,
DesktopFlowChallenge, DesktopIPCChallenge
  challenge_env.go         EnvironmentSetupChallenge,
EnvironmentTeardownChallenge
  challenge_mobile.go      MobileLaunchChallenge,
MobileFlowChallenge, InstrumentedTestChallenge

  -- Plugin and evaluators
  plugin.go                UserFlowPlugin (implements
plugin.Plugin)
  evaluators.go            12 assertion evaluators

  -- Infrastructure
  container_infra.go       TestEnvironment, PlatformGroup
  options.go               ChallengeOption functional options
  result_parser.go         ParseTestResultToValues,
ParseBuildResultToValues
  types.go                 TestResult, BuildResult, LintResult,
config types
```

Getting Started

1. Register the Plugin

The `UserFlowPlugin` must be initialized with an assertion engine before evaluators become available:

```go
import (
    "digital.vasic.challenges/pkg/assertion"
    "digital.vasic.challenges/pkg/plugin"
    "digital.vasic.challenges/pkg/userflow"
)
```

```
engine := assertion.NewDefaultEngine()
ufPlugin := &userflow.UserFlowPlugin{}

ctx := &plugin.PluginContext{
    Config: map[string]any{
        "assertion_engine": engine,
    },
}
if err := ufPlugin.Init(ctx); err != nil {
    log.Fatal(err)
}
```

2. Create an Adapter

Choose the adapter for your platform:

```
// API testing
api := userflow.NewHTTPAPIAdapter("http://localhost:8080")

// Browser testing via Playwright + CDP
browser := userflow.NewPlaywrightCLIAdapter("ws://localhost:9222")

// Go build/test
goBuild := userflow.NewGoCLIAdapter("/path/to/project")
```

3. Build a Challenge

Use a challenge template with your adapter:

```
challenge := userflow.NewAPIFlowChallenge(
    "CH-API-001",
    "Login and List Resources",
    "Verify login flow and resource listing",
    nil, // no dependencies
    api,
    userflow.APIFlow{
        Name: "login-and-list",
        Credentials: userflow.Credentials{
            Username: "admin",
            Password: "password",
            URL:      "http://localhost:8080",
        },
        Steps: []userflow.APIStep{
            {
                Name:           "list-items",
                Method:         "GET",
                Path:           "/api/v1/items",
                ExpectedStatus: 200,
            },
        },
```

```
    },
)
```

4. Register and Run

Register the challenge with the challenge registry and execute it through the runner as you would any other challenge.

Documentation Index

Document	Description
architecture.md	High-level architecture: adapter pattern, challenge templates, container infra, plugin system
framework-comparison.md	Comprehensive comparison of ALL framework adapters with feature matrix and selection guide
Core Adapters	
api-adapter.md	APIAdapter interface, HTTPAPIAdapter, WebSocket support
browser-adapter.md	BrowserAdapter interface, PlaywrightCLIAdapter, CDP connection
mobile-adapter.md	MobileAdapter interface, ADBCLIAdapter, device configuration
desktop-adapter.md	DesktopAdapter interface, TauriCLIAdapter, WebDriver protocol
build-adapter.md	BuildAdapter interface, Gradle/npm/Go/Cargo implementations
process-adapter.md	ProcessAdapter interface, ProcessCLIAdapter, lifecycle management
Browser Adapters	
selenium-adapter.md	SeleniumAdapter: W3C WebDriver protocol, Selenium Grid, multi-browser
cypress-adapter.md	CypressCLIAdapter: Cypress CLI spec generation, Chrome-focused
puppeteer-adapter.md	PuppeteerAdapter: Node.js scripts, CDP endpoint, container fallback
Mobile Adapters	
appium-adapter.md	AppiumAdapter: Appium 2.0, cross-platform Android/iOS, W3C extensions
maestro-adapter.md	MaestroCLIAdapter: YAML-driven flows, declarative mobile testing

Document	Description
espresso-adapter.md	EspressoAdapter: instrumented tests via Gradle + ADB hybrid
robolectric-adapter.md	RobolectricAdapter: JVM-based Android unit tests, no emulator
Protocol Adapters	
grpc-adapter.md	GRPCAdapter/GRPCCLIAdapter: grpcurl CLI, server reflection, streaming
websocket-adapter.md	WebSocketFlowAdapter: gorilla/websocket, bidirectional messaging
Guides	
challenge-templates.md	All challenge template types with constructor signatures
evaluators.md	All 12 evaluators with input types and pass conditions
container-integration.md	TestEnvironment, PlatformGroup, setup/teardown lifecycle
writing-challenges.md	Step-by-step guide for creating new challenges
writing-adapters.md	Step-by-step guide for implementing new platform adapters